



INGENIERÍA MECATRÓNICA

Programación Avanzada

Actividad: U3A2.

Etapa 2. Integración de Hardware

Alumnos:

Rubén Álvarez Durán 22030003

Samantha Elena Camacho Badillo 22030005

Juan David Castruita Castañeda 22030006

Ángel Daniel Cisneros Pérez 22030007

Docente: Osbaldo Aragón Banderas

Fecha: 2026-05-09

Índice

Contenido

1. Objetivo	1
2. Descripción del sistema	1
3. Metodología.....	2
Fase 1. Inventario y Selección de hardware	2
Fase 2. Diagrama de conexiones eléctricas	4
Fase 3. Desarrollo del sistema embebido	5
Fase 4. Pruebas unitarias	16
4. Diagrama de bloques	17
5. Comunicación implementada	18
6. Evidencias	19
7. Conclusiones técnicas	22

1. Objetivo

El objetivo de esta actividad es seleccionar, conectar y programar el hardware necesario para el desarrollo del proyecto de innovación que se ha venido desarrollando. El sistema AgriHome es una maceta inteligente autónoma, integrando sensores de humedad de suelo, intensidad lumínica y radiación UV con un microcontrolador ESP32-S3. El sistema permitirá adquirir datos del entorno de la planta, controlar actuadores como bomba de agua y luces LED UV, y preparar la base para una futura comunicación con aplicación móvil.

2. Descripción del sistema

En esta etapa, el proyecto de AgriHome se desarrolla de forma tangible mediante la integración del hardware principal. La maceta inteligente se basa en un microcontrolador ESP32-S3 que recibe información de sensores conectados al sistema y toma decisiones básicas de control.

El proyecto se enfoca en unir el mundo físico con el digital: los sensores detectan humedad de suelo, luz ambiental y radiación UV; el microcontrolador interpreta las señales; y los actuadores ejecutan acciones como encender la bomba de agua o activar iluminación UV. Todo esto con el fin de cumplir con el objetivo del proyecto, el cual es garantizar que se pueda cultivar un alimento vegetal a pequeña escala, orientado al consumo doméstico en zonas urbanas. Todo este sistema se desarrolla tomando en cuenta las necesidades inmediatas de los consumidores urbanos, permitiendo una facilitación del proceso de cultivación.

Las fases desarrolladas son:

- Selección y documentación del hardware del sistema.
- Diseño del esquema de conexiones eléctricas.
- Desarrollo del software embebido para lectura de sensores y control de actuadores.
- Pruebas unitarias de cada módulo hardware-software.

3. Metodología

Fase 1. Inventario y Selección de hardware

En la primera fase de este desarrollo, se establece el inventario que se requiere para el sistema. Se selecciona el hardware necesario, incluyendo todos los componentes que conforman los sistemas electrónico y estructural de AgriHome. Los componentes son los siguientes:

- ESP32-S3: Procesamiento principal y conectividad.
- Sensores capacitivos de humedad: medición del estado del sustrato.
- BH1750/GY-302: medición de intensidad luminica.
- GUVA-S12SD: medición de radiación UV.
- Bomba de agua 12V: sistema de riego.
- L298N: control de la bomba.
- LEDs UV: iluminación complementaria.
- PN2222A: control de LEDs UV.
- Estructura de madera y acrílico: soporte físico del sistema.

De igual forma, se implementa el lenguaje de programación C++ con framework Arduino en el entorno de desarrollo PlatformIO de Visual Studio Code. El inventario se visualiza de mejor manera con la siguiente tabla:

Componente	Tipo	Modelo	Interfaz	Cantidad
Microcontrolador	Procesamiento	ESP32-S3	UART	1
Sensor capacitivos de humedad	Medición de Humedad	Genérico Arduino	Analógico	2
Sensor de intensidad lumínica	Medición de intensidad lumínica	BH1750/GY-302	Analógico	1
Sensor de radiación UV	Medición de radiación UV	GUVA-S12SD	Analógico	1
Bomba de agua 12V	Sistema de riego	Bomba DC 12V	Actuador	1
Puente H	Control de la bomba	L298N	Controlador	1
LEDs UV	Iluminación complementaria	Tira LED de cultivo	Actuador	1

Transistor BJT	Control de LEDs UV	PN2222A	Controlador	1
Estructura	Soporte físico del sistema	Estructura de madera y acrílico	Estructura de soporte	1

Los componentes empleados se eligieron tomando en cuenta las necesidades técnicas del sistema. De igual forma, se consideraron costos, funcionalidad, practicidad y la escalabilidad futura del sistema. El microcontrolador ESP32-S3 se eligió tomando en cuenta su versatilidad y la capacidad de interconectar diferentes sensores para un procesamiento mayor al que ofrecen otros controladores. Al contar con 44 pines y permitir la conexión WiFi/BT, este microcontrolador es la mejor opción para el proyecto. Los sensores analógicos proporcionan datos más exactos que aquellos que son digitales, lo cual permite un control más eficiente a la hora de garantizar el bienestar de los cultivos. El puente H L298N permite el control de la bomba que requiere un voltaje mayor al seguro para el microcontrolador. Controlar la bomba de agua mediante el ESP32-S3 requiere el uso de medidas de seguridad eléctrica para evitar el daño a componentes, especialmente el microcontrolador. Otro actuador es la tira de LEDs UV que permiten la iluminación complementaria necesaria para el cuidado de los cultivos. El transistor PN2222A controla estos LED UV mediante un manejo de cargas en dispositivos de este tipo. Finalmente, la estructura de madera se implementa para contener todo el espacio de cultivo y control. Se le ha realizado mantenimiento para garantizar estabilidad y evitar el desgaste que puede provocar la humedad. Este mantenimiento consiste en la aplicación de barniz como recubrimiento protector, sellando y protegiendo a la estructura de madera contra la humedad, rayos UV y desgaste, factores que se tomaron en cuenta para planificar el desarrollo correcto del sistema integrado. Una sección de acrílico contiene el sistema integrado de control, el cual se compone del microcontrolador y las diferentes conexiones eléctricas que conforman el sistema. Este material garantiza organización, durabilidad y protección de cualquier daño o factor externo.

Fase 2. Diagrama de conexiones eléctricas

Después de establecer los componentes del sistema de AgriHome, se realizaron las conexiones necesarias para el funcionamiento adecuado. En la figura 1, se muestra un diagrama de las conexiones eléctricas del sistema.

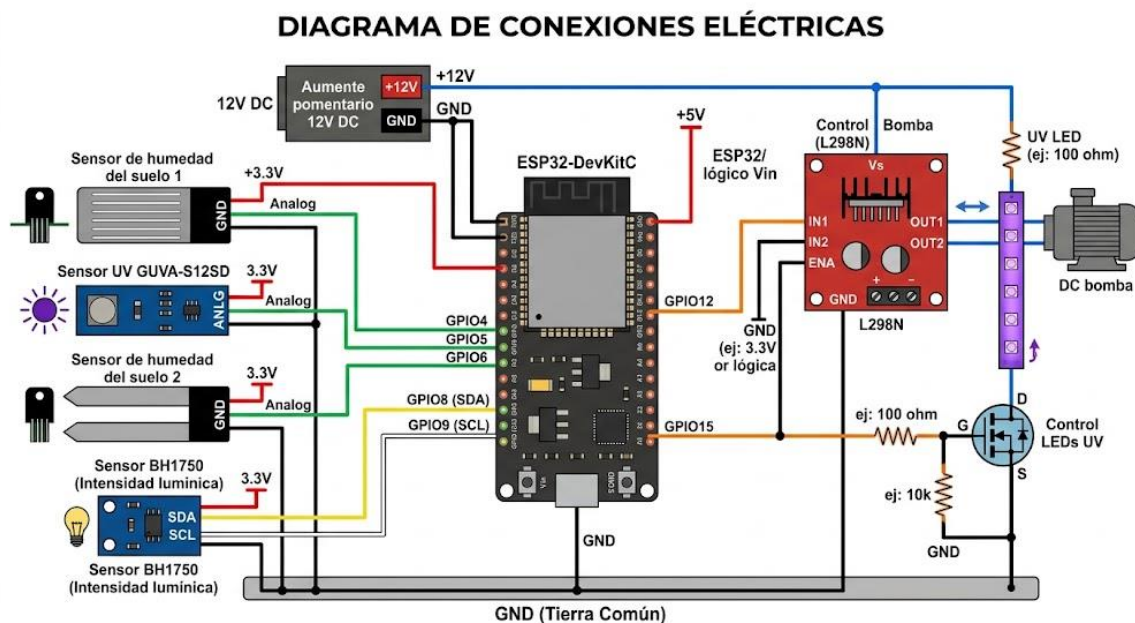


Figura 1. Diagrama de conexiones eléctricas del sistema integrado.

Las conexiones eléctricas entre el microcontrolador y los distintos sensores y actuadores, así como componentes que apoyan en el control como resistencias y transistores, se explican específicamente en la siguiente tabla.

Pin MCU	Nombre del Pin	Tipo	Componente	Señal
GPIO4	Medición de humedad	Input	Sensor de humedad 1	Analógica
GPIO5	Medición de radiación UV	Input	Sensor UV GUA-S12SD	Analógica
GPIO6	Medición de humedad	Input	Sensor de humedad 2	Analógica
GPIO8	Medición de intensidad lumínica	Input	SDA BH1750	I2C
GPIO9	Medición de intensidad lumínica	Input	SCL BH1750	I2C

GPIO12	Control de bomba	Output	Control de bomba mediante L298N	Digital
GPIO15	Iluminación complementaria	Output	Control LEDs UV mediante transistor	Digital

Todas las conexiones principales están conectadas a tierra común (GND). Se especifica, de igual forma, que se utilizan resistencias de 1k ohms y 10k ohms, para garantizar la seguridad eléctrica que los componentes requieren. Adicionalmente, el transistor PN2222A forma parte del control de la tira de los LED UV que conforman la parte de iluminación complementaria de los cultivos.

Fase 3. Desarrollo del sistema embebido

El firmware fue desarrollado en el entorno de desarrollo de PlatformIO en Visual Studio Code, utilizando el framework Arduino. Se implementaron los siguientes módulos:

- Lectura de sensores analógicos.
- Comunicación I2C con BH1750.
- Conversión ADC a voltaje.
- Control ON/OFF de actuadores.
- Generación de datos CSV.
- Comunicación Serial con Python.

Este sistema emplea una frecuencia de muestreo de 2 segundos. Los datos son recopilados y transmitidos en formato CSV, ejemplificado de la siguiente forma:

```
timestamp_ms,soil1_raw,soil1_v,soil2_raw,soil2_v,soil_avg_raw,uv_raw,uv_v,lux,pump_state,leds_state
```

En seguida se muestran los códigos de programación que se emplearon en el desarrollo del sistema AgriHome, siendo dos: uno en C++ para la programación del

microcontrolador ESP32-S3, y otro en Python para observar una interfaz gráfica en donde se visualizan gráficas de las mediciones de los distintos sensores.

Código para microcontrolador ESP32-S3:

```
// =====  
  
// PINES ESP32-S3  
  
// =====  
  
#define PIN_I2C_SDA    8  
  
#define PIN_I2C_SCL    9  
  
#define PIN_SOIL1_ADC  4  
  
#define PIN_UV_ADC     5  
  
#define PIN_SOIL2_ADC  6  
  
#define PIN_PUMP        12  
  
#define PIN_LEDS        15  
  
// =====  
  
// SENSOR BH1750  
  
// =====  
  
BH1750 lightMeter;  
  
bool bh1750_ok = false;  
  
// =====  
  
// FUNCIONES  
  
// =====  
  
float adcToVoltage(int raw) {  
    return (raw * 3.3f) / 4095.0f;  
}  
  
void imprimirEncabezadoCSV() {
```

```

Serial.println(

    "timestamp_ms,"

    "soil1_raw,soil1_v,"

    "soil2_raw,soil2_v,"

    "soil_avg_raw,"

    "uv_raw,uv_v,"

    "lux,"

    "pump_state,"

    "leds_state"

);

}

void setup() {

    Serial.begin(115200);

    delay(3000);

    analogReadResolution(12);

    analogSetPinAttenuation(PIN_SOIL1_ADC, ADC_11db);

    analogSetPinAttenuation(PIN_SOIL2_ADC, ADC_11db);

    analogSetPinAttenuation(PIN_UV_ADC, ADC_11db);

    pinMode(PIN_PUMP, OUTPUT);

    pinMode(PIN_LEDS, OUTPUT);


    digitalWrite(PIN_PUMP, LOW);

    digitalWrite(PIN_LEDS, LOW);

    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);

    Wire.setClock(100000);

```

```

delay(500);

bh1750_ok = lightMeter.begin(
    BH1750::CONTINUOUS_HIGH_RES_MODE,
    0x23,
    &Wire
);

Serial.println("# MACETA INTELIGENTE");
Serial.println("# MOD0 CSV ACTIVO");
Serial.print("# BH1750=");
Serial.println(bh1750_ok ? "OK" : "NO_DETECTADO");
imprimirEncabezadoCSV();
}

void loop() {
    unsigned long timestamp = millis();

    int soil1Raw = analogRead(PIN_SOIL1_ADC);
    int soil2Raw = analogRead(PIN_SOIL2_ADC);
    int uvRaw = analogRead(PIN_UV_ADC);

    float soil1V = adcToVoltage(soil1Raw);
    float soil2V = adcToVoltage(soil2Raw);
    float uvV = adcToVoltage(uvRaw);

    int soilAvgRaw = (soil1Raw + soil2Raw) / 2;

    float lux = -1.0;

    if (bh1750_ok) {
        lux = lightMeter.readLightLevel();
    }
}

```

```

    int pumpState = digitalRead(PIN_PUMP);

    int ledsState = digitalRead(PIN_LEDS);

    Serial.print(timestamp);

    Serial.print(",");

    Serial.print(soil1Raw);

    Serial.print(",");

    Serial.print(soil1V, 3);

    Serial.print(",");

    Serial.print(soil2Raw);

    Serial.print(",");

    Serial.print(soil2V, 3);

    Serial.print(",");

    Serial.print(soilAvgRaw);

    Serial.print(",");

    Serial.print(uvRaw);

    Serial.print(",");

    Serial.print(uvV, 3);

    Serial.print(",");

    Serial.print(lux, 2);

    Serial.print(",");

    Serial.print(pumpState);

    Serial.print(",");

    Serial.println(ledsState);

    delay(INTERVALO_MS);
}

```

Código Python:

```
import serial

import pandas as pd

import matplotlib.pyplot as plt

import time

import os

import sys

# =====

# CONFIGURACIÓN

# =====

PUERTO = "COM4"

BAUDIOS = 115200

MAX_MUESTRAS = 100

ARCHIVO_CSV = "datos_maceta.csv"

# =====

# ABRIR SERIAL

# =====

try:

    ser = serial.Serial()

    ser.port = PUERTO

    ser.baudrate = BAUDIOS

    ser.timeout = 2

    # No reiniciar automáticamente la ESP32

    ser.dtr = False

    ser.rts = False
```

```

        ser.open()

        time.sleep(2)
except Exception as e:

    print("ERROR ABRIENDO PUERTO SERIAL")

    print(e)

    print("\nRevisa:")

    print("- Que el puerto COM sea correcto.")

    print("- Que no esté abierto PlatformIO Monitor.")

    print("- Que el ESP32 esté conectado.")

    sys.exit()

# =====

# VARIABLES

# =====

lineas = []

header = None

print("\n=====")

print(" LECTURA DE SENSORES - MACETA")

print("=====")

print(f"Puerto: {PUERTO}")

print(f"Baudios: {BAUDIOS}")

print(f"Muestras objetivo: {MAX_MUESTRAS}")

print("-----")

print("Si no aparecen datos, presiona RESET/EN manualmente.")

print("=====\n")

# =====

```

```

# LECTURA SERIAL

# =====

try:

    while len(lineas) < MAX_MUESTRAS:

        try:

            linea = ser.readline().decode(errors="ignore").strip()

        except serial.SerialException as e:

            print("\nERROR SERIAL DURANTE LECTURA")

            print(e)

            ser.close()

            sys.exit(

        if not linea:

            continue

        if linea.startswith("#"):

            print(linea)

            continue

        if linea.startswith("timestamp_ms"):

            header = linea.split(",")

            print("\nCSV detectado correctamente.")

            print("Columnas:")

            for col in header:

                print(f" - {col}")

            print()

            continue

        if header and "," in linea:

```

```

partes = linea.split(",")

if len(partes) != len(header):

    print("Línea ignorada:")

    print(linea)

    continue

lineas.append(partes)

dato = dict(zip(header, partes))

os.system("cls" if os.name == "nt" else "clear")

print("=====")

print(" MACETA INTELIGENTE - LECTURA ACTUAL")

print("=====")

print(f"Muestra: {len(lineas)} / {MAX_MUESTRAS}")

print(f"Tiempo ESP32: {dato['timestamp_ms']} ms")

print("\n----- HUMEDAD DE SUELO -----")

print(f"Sensor suelo 1: {dato['soil1_raw']} RAW | {dato['soil1_v']}
V")

print(f"Sensor suelo 2: {dato['soil2_raw']} RAW | {dato['soil2_v']}
V")

print(f"Promedio suelo: {dato['soil_avg_raw']} RAW")

print("\n----- RADIACIÓN UV -----")

print(f"GUVA-S12SD: {dato['uv_raw']} RAW | {dato['uv_v']} V")

print("\n----- INTENSIDAD LUMINICA -----")

print(f"BH1750: {dato['lux']} lux")

print("\n----- ACTUADORES -----")

print(f"Bomba: {'ENCENDIDA' if dato['pump_state'] == '1' else
'APAGADA'}")

```

```

        print(f"LEDs: {'ENCENDIDOS' if dato['leds_state'] == '1' else
'APAGADOS'}")

        print("\n=====")

except KeyboardInterrupt:

    print("\nCaptura detenida manualmente con Ctrl+C.")

finally:

    if ser.is_open:

        ser.close()

if not lineas:

    print("\nNo se recibieron datos válidos.")

    sys.exit()

# =====

# CREAR CSV

# =====

df = pd.DataFrame(lineas, columns=header)

for col in df.columns:

    df[col] = pd.to_numeric(df[col], errors="coerce")

df.to_csv(ARCHIVO_CSV, index=False)

print(f"\nCSV guardado correctamente: {ARCHIVO_CSV}")

# =====

# GRÁFICA HUMEDAD DE SUELO

# =====

plt.figure()

plt.plot(df["timestamp_ms"], df["soil1_raw"], label="Humedad suelo 1")

plt.plot(df["timestamp_ms"], df["soil2_raw"], label="Humedad suelo 2")

plt.plot(df["timestamp_ms"], df["soil_avg_raw"], label="Promedio suelo")

```

```

plt.xlabel("Tiempo (ms)")

plt.ylabel("ADC RAW")

plt.title("Humedad de suelo")

plt.legend()

plt.grid(True)

plt.savefig("grafica_humedad_suelo.png", dpi=300)

plt.show()

# =====

# GRÁFICA UV

# =====

plt.figure()

plt.plot(df["timestamp_ms"], df["uv_raw"])

plt.xlabel("Tiempo (ms)")

plt.ylabel("ADC RAW")

plt.title("Radiación UV GUVAS-S12SD")

plt.grid(True)

plt.savefig("grafica_uv.png", dpi=300)

plt.show()

# =====

# GRÁFICA LUZ

# =====

plt.figure()

plt.plot(df["timestamp_ms"], df["lux"])

plt.xlabel("Tiempo (ms)")

plt.ylabel("Lux")

```

```

plt.title("Intensidad luminica BH1750")

plt.grid(True)

plt.savefig("grafica_lux.png", dpi=300)

plt.show()

# =====

# GRÁFICA ACTUADORES

# =====

plt.figure()

plt.plot(df["timestamp_ms"], df["pump_state"], label="Bomba")

plt.plot(df["timestamp_ms"], df["leds_state"], label="LEDs")

plt.xlabel("Tiempo (ms)")

plt.ylabel("Estado 0/1")

plt.title("Estado de actuadores")

plt.legend()

plt.grid(True)

plt.savefig("grafica_actuadores.png", dpi=300)

plt.show()

print("\nARCHIVOS GENERADOS:")

print(f"- {ARCHIVO_CSV}")

print("- grafica_humedad_suelo.png")

print("- grafica_uv.png")

print("- grafica_lux.png")

print("- grafica_actuadores.png")

```

Fase 4. Pruebas unitarias

Las pruebas realizadas fueron las siguientes:

- HW-01: Prueba de carga del firmware.
- HW-02: Lectura sensor humedad 1.
- HW-03: Lectura sensor humedad 2.
- HW-04: Lectura BH1750.
- HW-05: Lectura sensor UV.
- HW-06: Activación bomba.
- HW-07: Activación LEDs UV.
- HW-08: Generación de CSV y gráficas.

Después de realizar estas pruebas en secuencia, se han recopilado los resultados obtenidos:

- Lecturas seriales correctas.
- CSV generado correctamente.
- Gráficas generadas mediante Python.
- Sensores funcionales.
- Actuadores listos para integración final.

4. Diagrama de bloques

El funcionamiento en este sistema se representa en la figura 2, la cual muestra un diagrama de bloques que explica la lógica con la que opera el sistema. Se relacionan los distintos componentes que conforman el sistema, y se muestra la comunicación con el microcontrolador y el procesamiento que éste realiza para controlar los actuadores necesarios.

Los sensores miden las condiciones ambientales en las que se encuentran los cultivos del usuario. Con los datos medidos, el microcontrolador realiza las operaciones necesarias para asegurar el cuidado de los cultivos. Se determina el control mediante instrucciones en donde el sistema determina si es o no necesario activar los actuadores que realizan el cuidado de los cultivos. Estos actuadores son la bomba de agua que conforma el sistema de riego, así como la tira de LEDs UV que complementan la iluminación que recibe la planta.

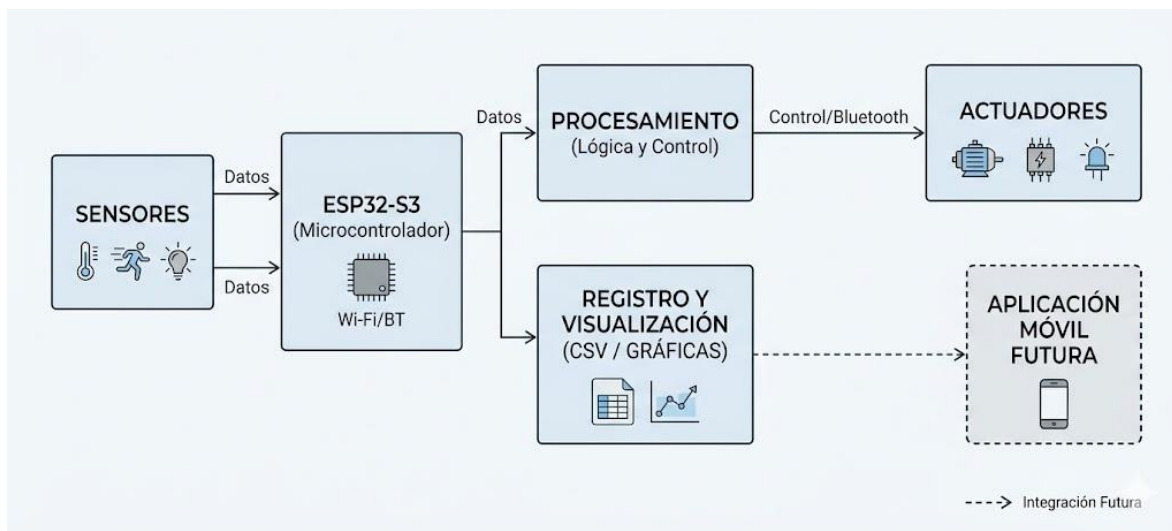


Figura 2. Diagrama de bloques del sistema de AgriHome.

5. Comunicación implementada

La comunicación implementada en el sistema de AgriHome es de tipo UART/Serial a 115200 baudios, la cual se realiza con comunicación entre el microcontrolador ESP32 y Python.

La comunicación I2C se realiza para:

- BH1750, sensor de intensidad lumínica
- Dirección 0x23

Mientras que la conversión ADC se realiza para:

- Sensores analógicos de humedad y UV.

El microcontrolador ESP32-S3 proporciona datos, los cuales se guardan en una serie de archivos CSV y PNG; estos últimos muestran gráficas de los distintos sensores que se emplean, mientras que, adicionalmente, estos datos se almacenan en un archivo CSV para tener un registro histórico de los datos y las mediciones obtenidas de los sensores.

6. Evidencias

En las figuras 3 a 7 se muestran fotografías del armado físico del sistema, así como las gráficas que muestran las mediciones de los sensores y Python.

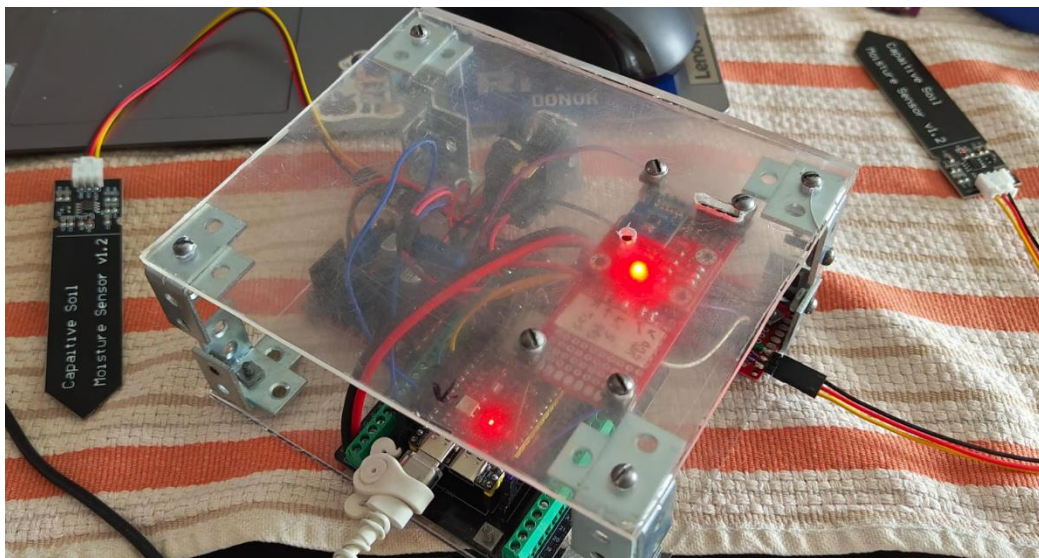


Figura 3. Sistema integrado en su respectiva estructura de acrílico.

```
=====
MACETA INTELIGENTE - LECTURA ACTUAL
=====
Muestra: 100 / 100
Tiempo ESP32: 201737 ms

----- HUMEDAD DE SUELO -----
Sensor suelo 1: 2663 RAW | 2.146 V
Sensor suelo 2: 2404 RAW | 1.937 V
Promedio suelo: 2533 RAW

----- RADIACIÓN UV -----
GUVA-S12SD: 0 RAW | 0.000 V

----- INTENSIDAD LUMINICA -----
BH1750: 20.83 lux

----- ACTUADORES -----
Bomba: APAGADA
LEDs: APAGADOS
=====
```

Figura 4. Lectura de sensores mostrada en Python.

```

=====
Muestra: 100 / 100
Tiempo ESP32: 201738 ms

----- HUMEDAD DE SUELO -----
Sensor suelo 1: 2667 RAW | 2.149 V
Sensor suelo 2: 2410 RAW | 1.942 V
Promedio suelo: 2538 RAW

----- RADIACIÓN UV -----
GUVA-S12SD: 0 RAW | 0.000 V

----- INTENSIDAD LUMINICA -----
BH1750: 22.50 lux

----- ACTUADORES -----
Bomba: APAGADA
LEDs: APAGADOS

=====

CSV guardado correctamente: datos_maceta.csv

ARCHIVOS GENERADOS:
- datos_maceta.csv
- grafica_humedad_suelo.png
- grafica_uv.png
- grafica_lux.png
- grafica_actuadores.png

```

Figura 5. Lectura de datos y archivos de registro que se generan.

```

--- Terminal on COM4 | 115200 8-N-1
--- Available filters and text transformations: debug, default, direct, esp32_exception_decoder, hexlify, log2file, nocontrol,
l, printable, send_on_enter, time
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
# MACETA INTELIGENTE
# MODO CSV ACTIVO
# BH1750=OK
timestamp_ms,soil1_raw,soil1_v,soil2_raw,soil2_v,soil_avg_raw,uv_raw,uv_v,lux,pump_state,leds_state
3638,2659,2.143,2379,1.917,2519,0,0.000,0.00,0,0
5640,2659,2.143,2382,1.920,2520,0,0.000,19.17,0,0
7641,2655,2.140,2383,1.920,2519,0,0.000,19.17,0,0

```

Figura 6. Lectura de datos crudos, almacenados en archivo CSV.

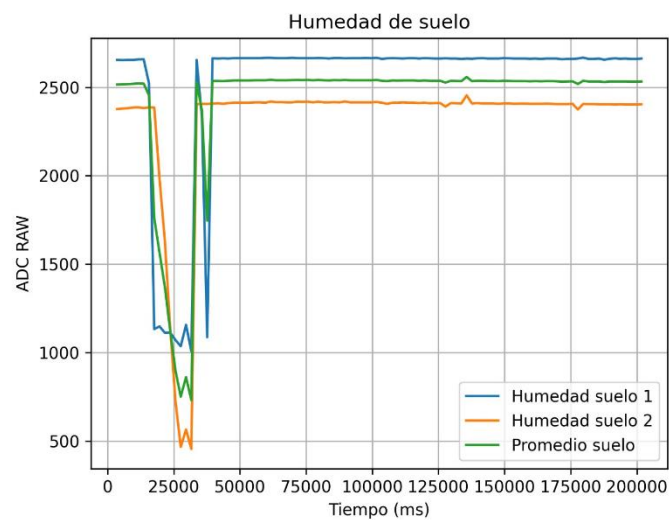


Figura 7. Gráfica de los sensores de humedad, agregando un promedio de ambos.

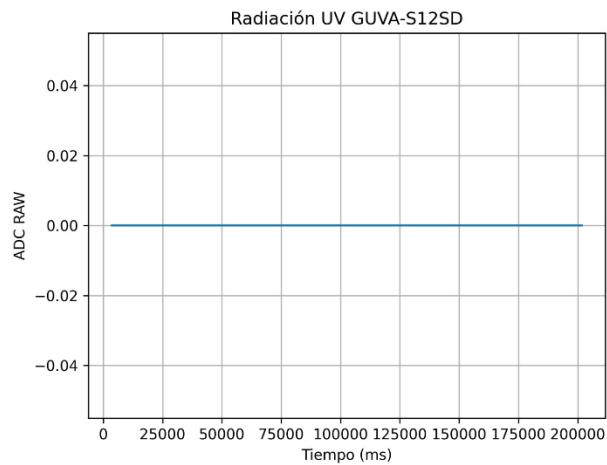


Figura 8. Gráfica de radiación (Baja por condiciones nubladas).

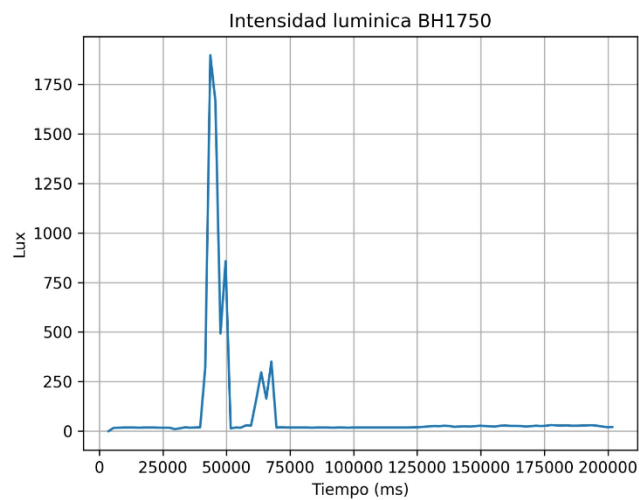


Figura 9. Gráfica de sensor BH1750.

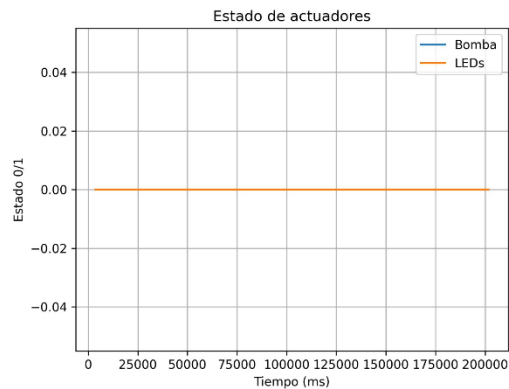


Figura 10. Gráfica del estado de actuadores (apagados).

7. Conclusiones técnicas

Durante esta etapa se logró integrar el hardware principal de la maceta inteligente autónoma. El ESP32-S3 fue capaz de leer sensores de humedad, intensidad lumínica y radiación UV, además de generar registros CSV y gráficas para análisis de datos. El sistema establece una base funcional para integrar posteriormente una aplicación móvil y automatización más avanzada.

Este sistema aún tiene potencial para continuar desarrollándose, tomando en cuenta que la automatización de los actuadores puede ser optimizada analizando los requerimientos específicos de los cultivos, así como las necesidades de los usuarios. Puede ser optimizado con una mejor estructura, así como sensores más exactos y actuadores más eficientes. La estructura puede ser mejorada para garantizar una protección más estable a los componentes que conforman el sistema integrado de AgriHome. Asimismo, este proyecto se puede complementar con una aplicación que permita un control más eficiente y una interacción más efectiva con el usuario, permitiendo que éste monitoree sus cultivos a distancia o que ésta aplicación le proporcione información útil al usuario. Actualmente, el sistema se ha desarrollado de forma correcta, evaluando que los sensores funcionan correctamente, proporcionando mediciones cruciales para el control del sistema de cultivo. De igual forma, estas mediciones y datos se han registrado de forma eficiente en archivos CSV, además de que el sistema ha proporcionado gráficas. Estos productos resultarán indispensables para realizar un análisis de datos, lo cual nos permite comprender de mejor manera el funcionamiento del sistema, así como planificar futuras expansiones y un desarrollo más productivo.